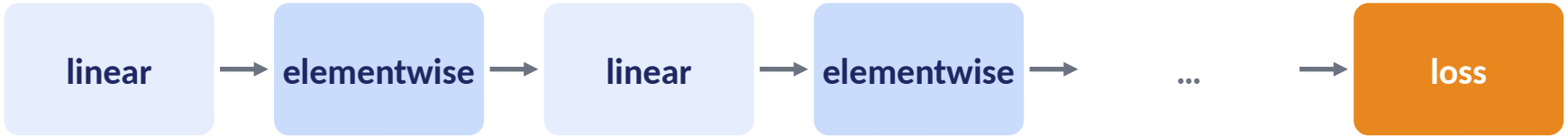
Mathematical Foundations for Data Science & Computational Intelligence

The Gradient in One Pass

Module 5 · Multivariable Calculus · Session 5.4

Mohsin Raza · Fall 2026

Remember This Product?



$$J = J_{\text{loss}} \cdot \text{diag}(\sigma') \cdot W_k \cdots \text{diag}(\sigma') \cdot W_2 \cdot \text{diag}(\sigma') \cdot W_1$$

Today it explains two things: how a computer evaluates it cheaply, and why it sometimes breaks.

Where You'll Meet This at Work

1 `loss.backward()`

Every autodiff library (PyTorch, JAX, TensorFlow, R's torch) applies exactly the chain rule, mechanically, to a graph of operations.

2 Debugging training

NaNs, stalls, a model that won't learn. The question is almost always: what happened to the gradient on its way back?

Libraries spare you from computing gradients by hand. They don't spare you from understanding them.

The Cost of Nudging

Model	Parameters	Forward runs to nudge every knob	At 1 ms per run
Toy model	1	2	instant
ResNet-50 (images)	~25.6 M	~25.6 M	~7 hours
GPT-2 small	~124 M	~124 M	**~34 hours**

1 pass

one backward pass: every gradient at once, for about the cost of 2–3 forward runs

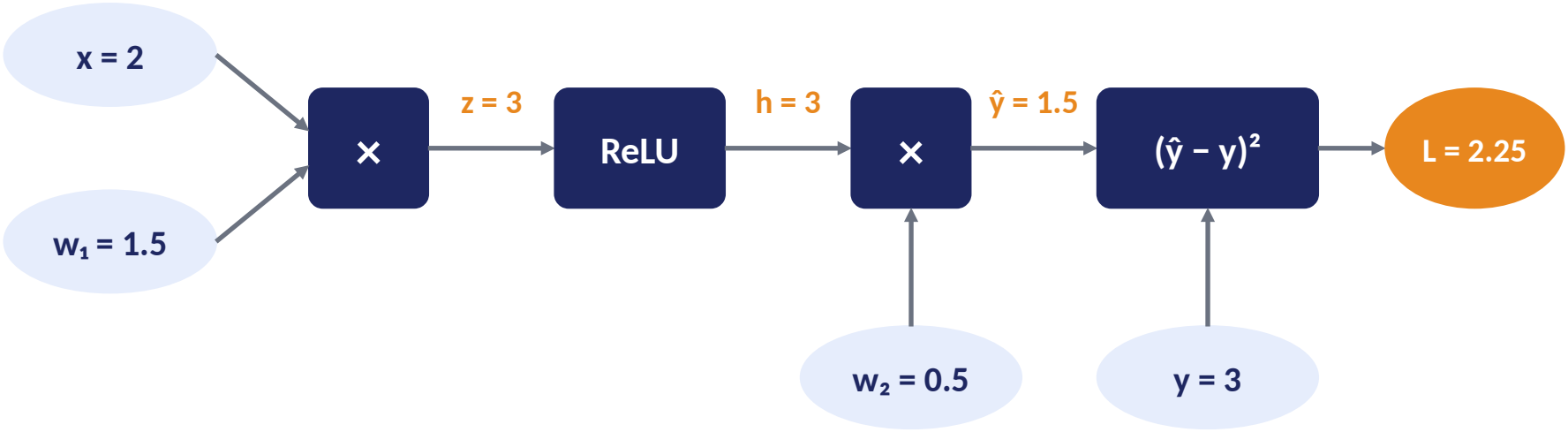
Session 5.1: nudging needs one extra forward run per knob, ~34 hours per step. Today: how one backward pass replaces all of them.

Illustrative: assumes 1 ms per forward evaluation; not measured runtimes.

Today

1. Computation graphs
2. Backprop by hand
3. Why backward is cheap
4. Vanishing & exploding gradients
5. The fixes
6. Module wrap & what's due

Any Computation Is a Graph



Nodes are operations; edges carry values. Each node has a derivative you learned in Session 5.1. The chain rule combines them along the edges.

Two Passes Over the Same Graph

Forward (left → right)

Compute and store every intermediate value.

Backward (right → left)

Multiply each node's local derivative onto the signal coming from the right.

Each node only needs its own local derivative. Backward reuses what forward stored, which is why training needs memory.

The Forward Pass

$$x = 2,\quad y = 3,\quad w_1 = 1.5,\quad w_2 = 0.5$$

Step	Computation	Value
$z = w_1 x$	1.5 × 2	**3**
$h = \mathrm{ReLU}(z)$	max(0, 3)	**3**
$\hat{y} = w_2 h$	0.5 × 3	**1.5**
$L = (\hat{y} - y)^2$	(1.5 − 3)²	**2.25**

Stored
z, h and ŷ are kept for the backward pass.

The Backward Pass

Step	Rule	Value
$\partial L/\partial \hat{y}$	$2(\hat{y} - y)$	−3
$\partial L/\partial w_2$	$\partial L/\partial \hat{y} \cdot h$	**−9**
$\partial L/\partial h$	$\partial L/\partial \hat{y} \cdot w_2$	−1.5
$\partial L/\partial z$	$\partial L/\partial h \cdot \mathrm{ReLU}'(z)$	−1.5
$\partial L/\partial w_1$	$\partial L/\partial z \cdot x$	**−3**

Each row: what came from the right × my own local derivative.

With a bias

$$z = w_1 x + b_1$$

$$\partial L/\partial b_1 = \partial L/\partial z \cdot 1$$

A bias is a weight whose input is always 1.

Does It Make Sense?

$$\partial L/\partial w_2 = -9$$

$$\partial L/\partial w_1 = -3$$

Increase both weights

Both gradients are negative, so the loss falls as the weights grow. The prediction (1.5) is below the truth (3): a bigger output is exactly what we need. ✓

The Whole Trick: Reuse

Step	Rule	Value
$\partial L/\partial \hat{y}$	$2(\hat{y} - y)$	−3
$\partial L/\partial w_2$	$\partial L/\partial \hat{y} \cdot h$	−9
$\partial L/\partial h$	$\partial L/\partial \hat{y} \cdot w_2$	−1.5
$\partial L/\partial z$	$\partial L/\partial h \cdot \mathrm{ReLU}'(z)$	−1.5
$\partial L/\partial w_1$	$\partial L/\partial z \cdot x$	−3

Computed once, used twice.

Each intermediate gradient is computed once and shared by everything upstream of it.

Nudging recomputes everything for every knob. Backprop computes each piece once.

Check It by Nudging

$$w_1 \to 1.51: \quad L = 2.2201, \quad \Delta L \approx -0.0299 \approx -3 \times 0.01 \checkmark$$

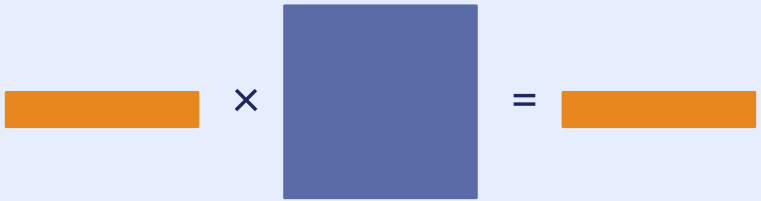
$$w_2 \to 0.51: \quad L = 2.1609, \quad \Delta L \approx -0.0891 \approx -9 \times 0.01 \checkmark$$

Same answers. The backward pass got both in one sweep; nudging needed a separate run per weight.

Same Product, Two Orders

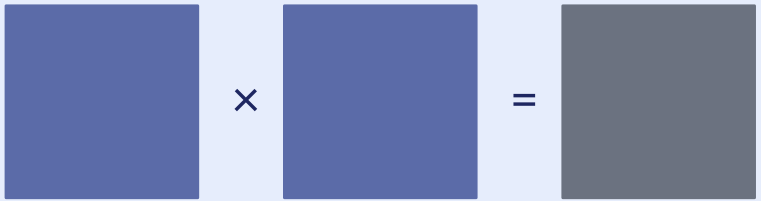
$$\nabla L \;=\; J_L \cdot J_k \cdot \cdots \cdot J_1 \qquad (J_L \text{ is a single row: the loss is one number})$$

From the left (backward)



row × matrix → row. Every intermediate stays a vector.

From the right (full matrices)



matrix × matrix → matrix. Every intermediate is a full matrix.

Forward-mode autodiff avoids these matrices too, but it pushes one input direction per pass: one pass per parameter.

Same product (matrix multiplication is associative). With one loss, starting from it is cheapest: that's reverse mode, i.e. backprop.

Backward Means Transpose



$$\nabla_{\text{input}} \mathrm{L} = \mathrm{J}^{\mathrm{T}} \cdot \nabla_{\text{output}} \mathrm{L}$$

The same rule, written for column gradients.

How Much Cheaper?

k layers of width n = 1000:

backward

$k \times 10^6$

vector–matrix: ~k·n²

multiplying full Jacobians

$k \times 10^9$

matrix–matrix: ~k·n³

About a thousand times cheaper than multiplying full Jacobians.

For a scalar loss, reverse mode gets the full gradient in one backward sweep, at a cost comparable to a few forward evaluations. Forward mode gets directional derivatives cheaply, but needs many sweeps for many parameters.

QUESTION FOR THE ROOM

Each layer multiplies the gradient by about 0.5. What's left after 20 layers?

And if each factor is about 2?

Remember 0.25?

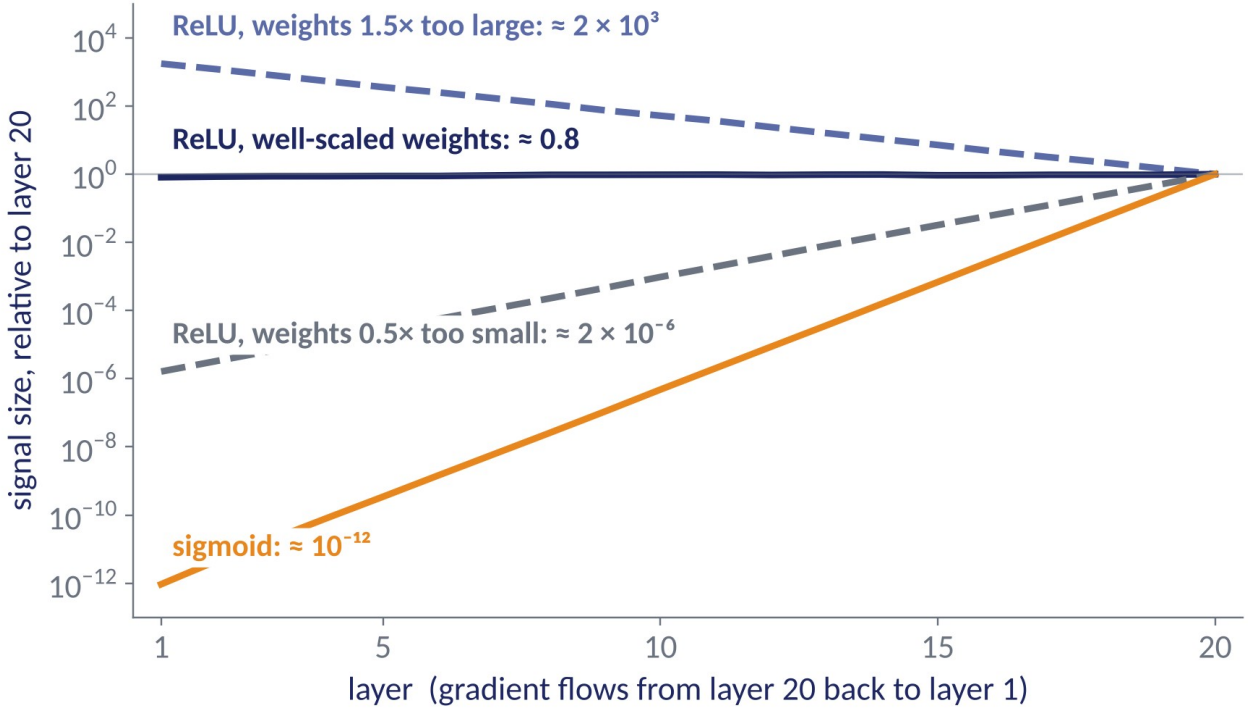
0.25

the largest σ′ can ever be (Session 5.1)

Sigmoid layers	Activation factors alone, at most
10	$0.25^{10} \approx 10^{-6}$
20	$0.25^{20} \approx 10^{-12}$

That's the activations' share, setting the weights aside. Unless the weights make up for it, the first layers barely learn.

What It Looks Like in a Real Network



Same ReLU network, different starting weights: stable, exploding or vanishing.

It's the product of everything, not just the activation.

20 layers · width 64 · average of 50 random networks

Day 1, Again: Overflow and Underflow

Exploding

numbers overflow → Inf, then NaN

"Loss went to NaN at step 3,000"

Vanishing

updates so small they're lost in floating-point rounding

"Training is stuck"

Day 1: numbers leaving the set a machine can represent. Here it is, inside training.

The Matrix View

Each layer's Jacobian stretches or shrinks vectors by its singular values (Module 3).

If every layer's largest singular value is below 1, the product must shrink. If every layer's smallest singular value is above 1, it must grow. In between, it depends on how the directions line up.

"Gradient flow" here means how the gradient signal flows back through the layers. The same phrase also names a continuous-time view of gradient descent (Module 8).

Fix the Factors

1. **ReLU instead of sigmoid**

   ReLU′ = 1 for active units: no 0.25 per layer.

2. **Careful initialization**

   Start with weights that neither shrink nor stretch (the flat line).

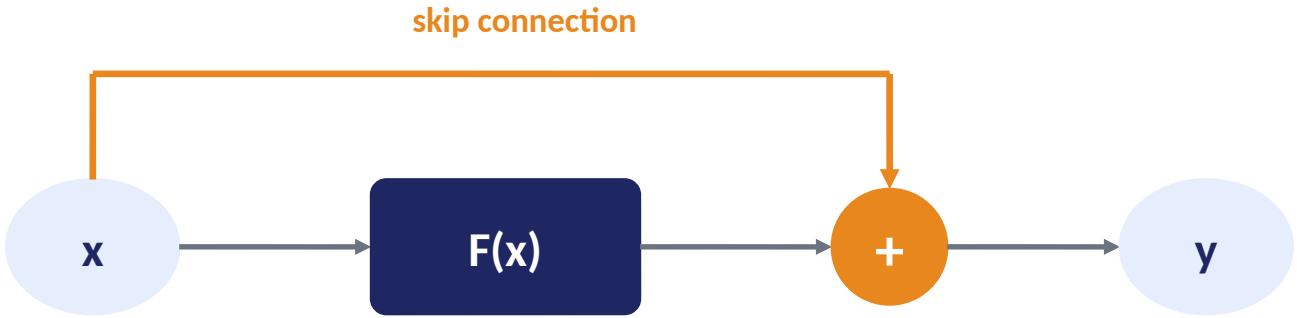
3. **Residual connections**

   Give the gradient a highway: next slide.

4. **Gradient clipping**

   Cap the gradient's size before updating: the standard defence against explosion.

Residual Connections: A Gradient Highway



$$J = I + J_F$$

a direct path for the gradient

$y = x + F(x)$. Even if J_F is tiny, $I + J_F$ isn't, so the gradient isn't forced to shrink. It doesn't guarantee stability: products of $I + J_F$ can still grow, so residuals come with normalization and careful initialization. Every transformer is built on this (Module 11).

A ReLU Can Die

Rerun the demo with $w_1 = -1.5$:

z	h	$\hat{y}$	L	$\partial L/\partial w_1$	$\partial L/\partial w_2$
−3	0	0	9	**0**	**0**

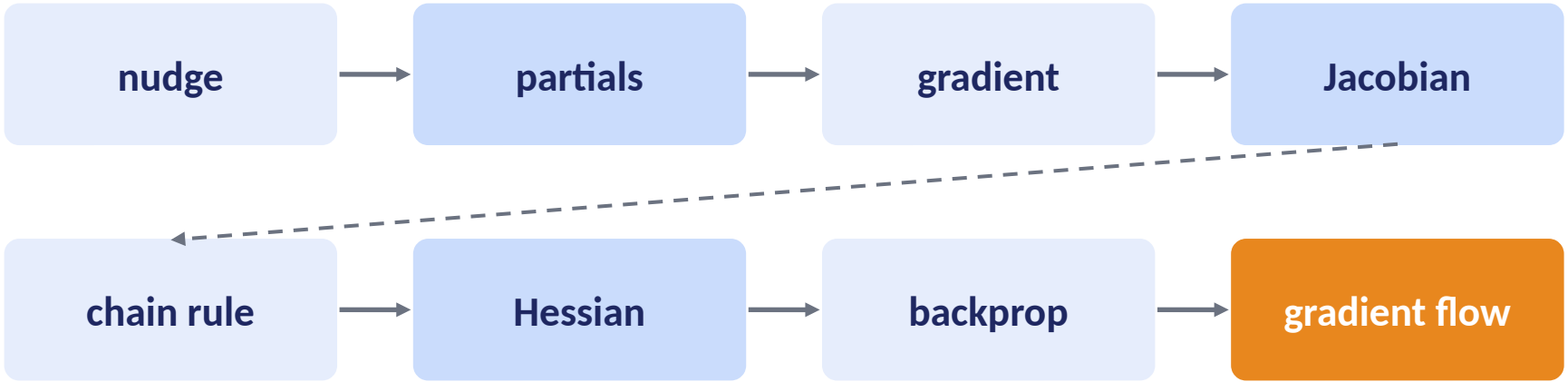
For this input, both weight gradients are 0. If the unit stays inactive for every training example, its incoming weights get no gradient through it, and it may stay off for good: the dead-ReLU problem.

That's why leaky ReLU exists (Part A, A3.8).

Choosing activations, initialization and residual structure is what architecture work actually is.



Module 5 in One Line



Two weeks ago: "which way do I turn the knob?" Now: which way, how the curvature bends the answer, and how a network gets it all in one pass.

Next: Reasoning Under Uncertainty

Module 6 · Probability, Clustering & Graphical Models

Questions?

The Dead ReLU, Worked in Full

$$w_1 = -1.5: \quad z = -3, \;\; h = 0, \;\; \hat{y} = 0, \;\; L = 9$$

$$\partial L/\partial \hat{y} = -6 \qquad \partial L/\partial w_2 = -6 \cdot h = 0$$

$$\partial L/\partial h = -6 \cdot 0.5 = -3 \qquad \partial L/\partial z = -3 \cdot \mathrm{ReLU}'(-3) = 0 \qquad \partial L/\partial w_1 = 0$$

Forward vs Reverse Mode

Forward mode carries a derivative forward per input: cheap when there are few inputs.

Reverse mode carries one signal backward per output: cheap when there is one loss.

Networks have millions of inputs (parameters) and one loss, so reverse mode wins.

Why the Identity Path Keeps Gradients Alive

When J_F is small, the singular values of $I + J_F$ are close to 1. That generally carries the gradient much better than multiplying many small J_F's. But small contractions or expansions can still add up over many layers: $(0.99)^{100} \approx 0.37$.

How the Simulation Was Run

20 layers, width 64, random Gaussian weights (sigmoid: variance 1/n; ReLU: variance 2/n, then scaled 1.5× or 0.5×).

One random input, a unit-length gradient at the top, averaged over 50 random networks.

Plotted: the size of $\partial L/\partial z$ at each layer, relative to layer 20. Exact values vary with the random seed; the orders of magnitude don't.